

SCRIBE

Loop Engineering in Long Horizon Tasks Needs Agent to Write What They Did Each Turn

周浩洋 · Jilin University (JLU), School of Software (SE)

Research Assistant · 2025.04.17 – Present

目录

01

training-free 方法

不可能实现 loop engineering 的
token 效率提升

02

training 方法 · ReAct

使用 ReAct 进行 loop engineering

03

ReAct 的问题

使用 ReAct 进行 loop engineering 的
问题

04

training 方法 · SCRIBE

使用 SCRIBE 进行 loop engineering

05

SCRIBE 上下文管理机制

细述 turn 级上下文与 KV-cache 复用

06

reward mechanism of a
turn

16 个 metric 与权重配比

07

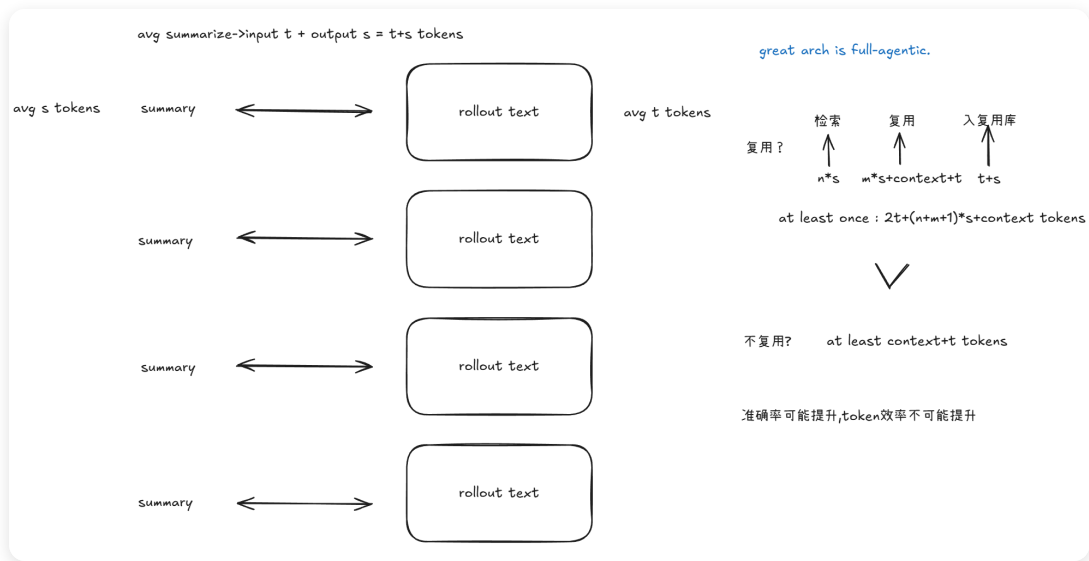
abstract of our contribution

小改动 → 上下文机制变化 → 衍生收益

01 · training-free 方法

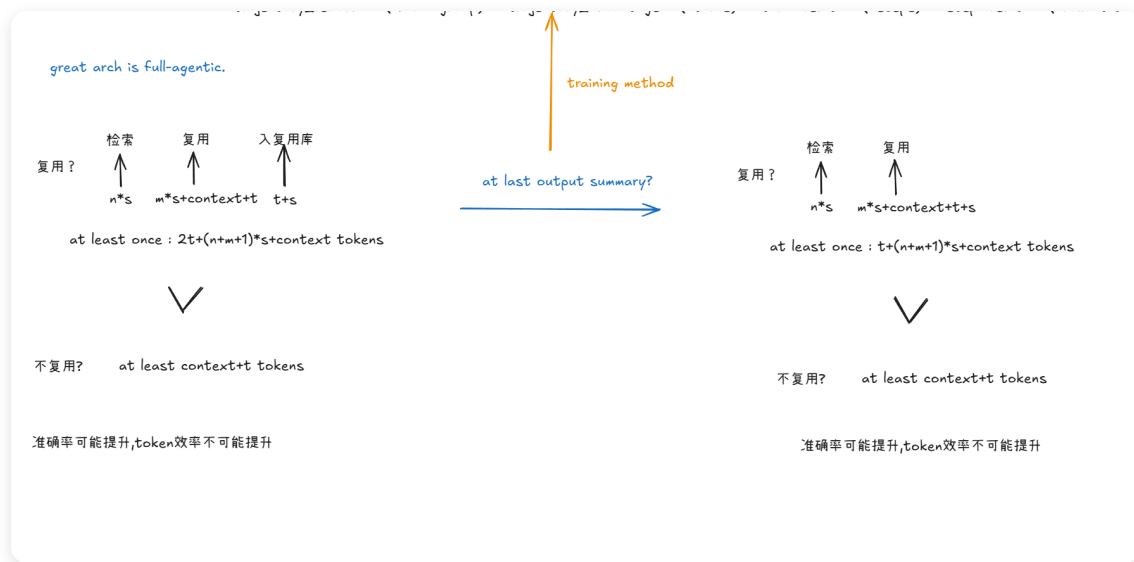
不可能实现 loop engineering 的 token 效率提升

01 · training-free 方法 — 不可能实现 loop engineering 的 token 效率提升



training-free 路径下, loop engineering 的 token 效率提升瓶颈 (一)

01 · training-free 方法 — 不可能实现 loop engineering 的 token 效率提升

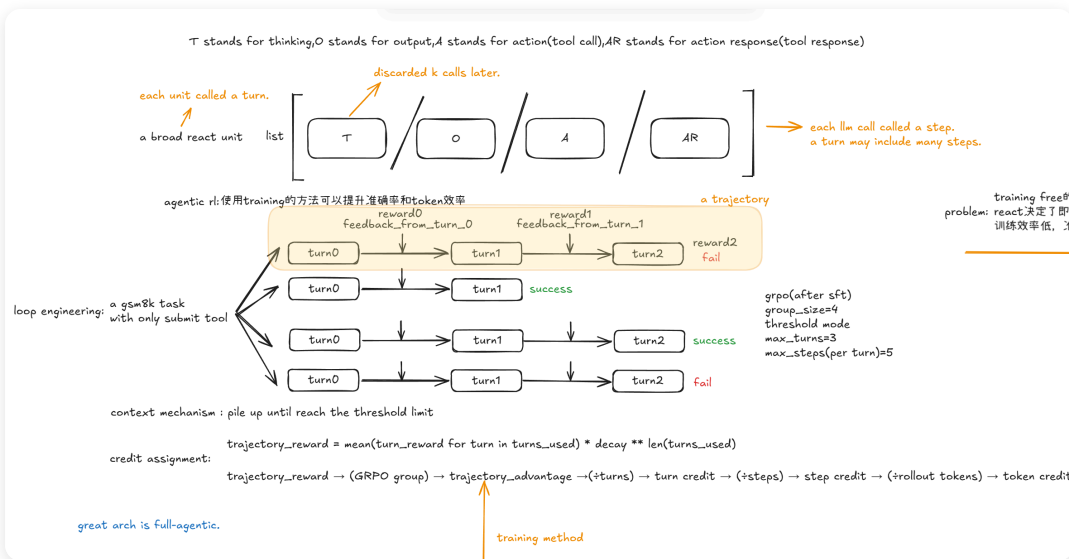


training-free 路径下, loop engineering 的 token 效率提升瓶颈 (二)

02 · training 方法

使用 ReAct 进行 loop engineering

02 · training 方法 — 使用 ReAct 进行 loop engineering



broad ReAct: 经典 Think→Act→Observe 循环作为 loop engineering 的训练范式

03 · ReAct 的问题

使用 ReAct 进行 loop engineering 的问题

03 · training 方法 — 使用 ReAct 进行 loop engineering 的问题

training free的话, 每次试错的经验是一次性的, 否则设计记忆架构, token效率必定增加。
problem: react决定了即使training, 状态和状态转换是隐式的, 对状态转换的后果没有前瞻性和规划, 短视, 所以要不断试错。
训练效率低, 准确率和token效率提高还不够多。且刚才的方法并没有精细的信用分配。

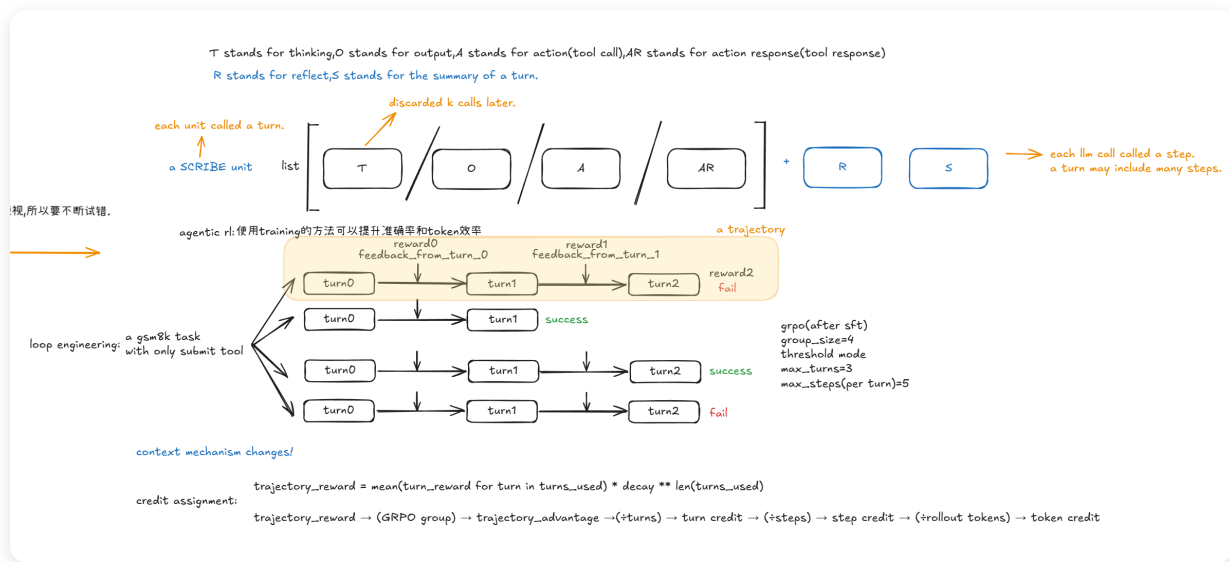


broad ReAct 在长程 loop engineering 中暴露的问题

04 · training 方法 · SCRIBE

ReAct 问题的解法：使用 SCRIBE 进行 loop engineering

04 · training 方法 — 使用 SCRIBE 进行 loop engineering

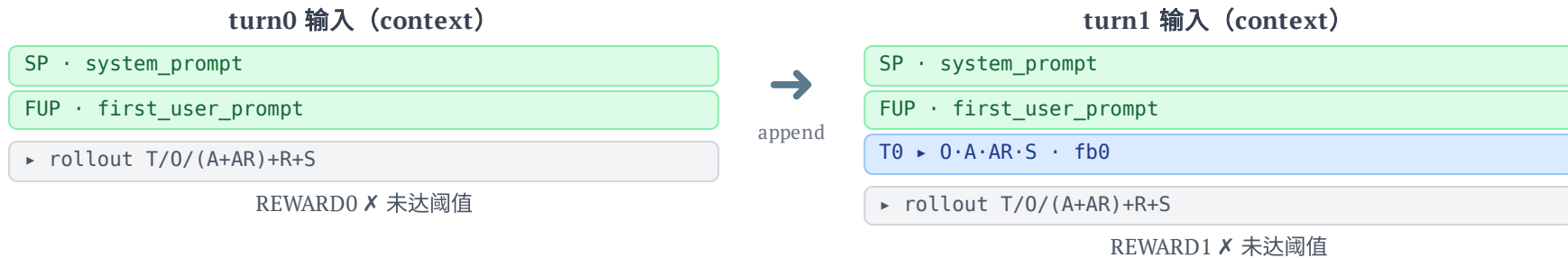


SCRIBE：在 ReAct 基础上为每个 turn 显式写入 Reflect (R) 与 Turn-Summary (S)

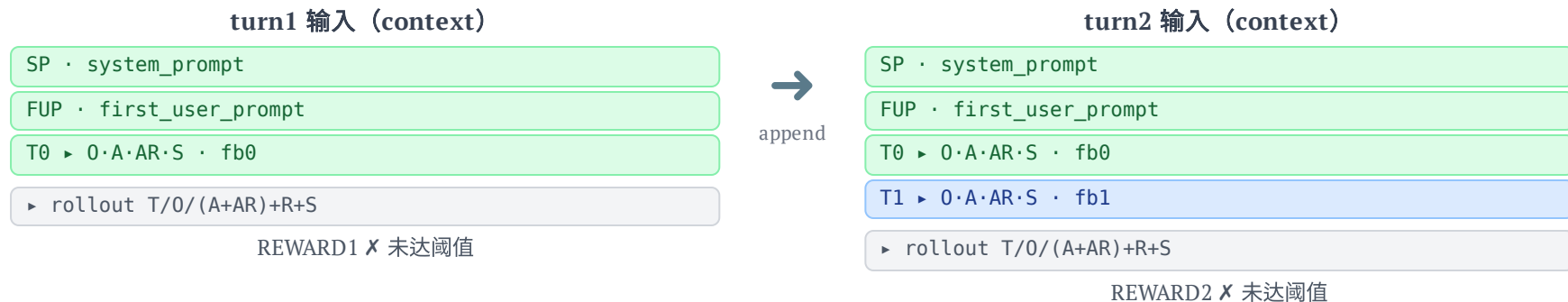
05 · SCRIBE 上下文管理机制

细述 turn 级上下文管理：KV-cache 复用、recent-k 保留、自动压缩

05 · SCRIBE 上下文管理机制 - turn0 → turn1



05 · SCRIBE 上下文管理机制 - turn1 → turn2



前缀 **SP + FUP + T0** 全部复用，仅追加 turn1 留存块 + fb1。

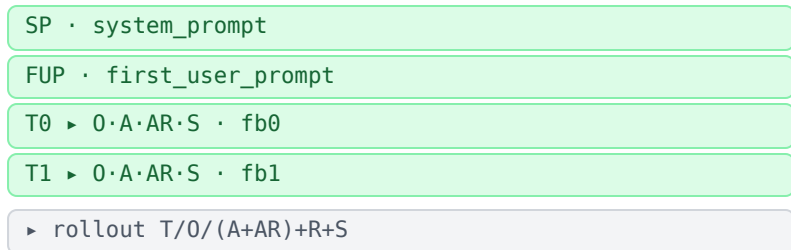
绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算

05 · SCRIBE 上下文管理机制 - turn2 → turn3

turn2 输入 (context)



REWARD2 ✗ 未达阈值



append

turn3 输入 (context)



REWARD3 ✗ 未达阈值

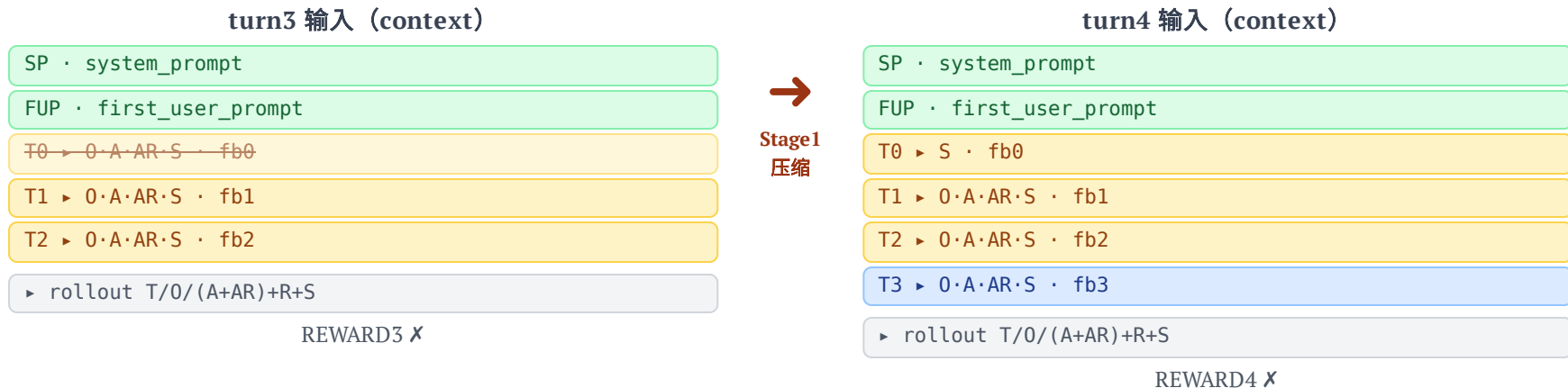
recent-k=3 窗口内逐 turn 追加，整段前缀 SP+FUP+T0+T1 复用，仅新增 T2。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算

05 · SCRIBE 上下文管理机制 - turn3 → turn4（首次压缩）



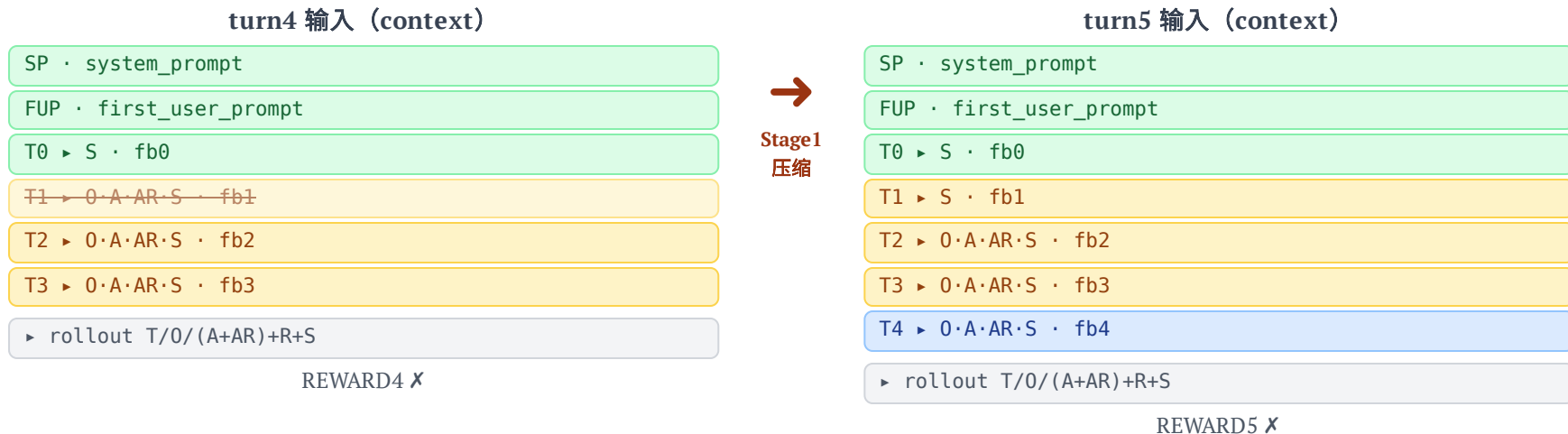
turn0 超出 recent-k=3 被降级为 S (O/A/AR 丢弃, 只留 turn_summary)；公共前缀在 **T0 处断裂** → T1/T2 虽内容不变也须重算。代价是换得后续 append 不再无限增长。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn4 → turn5（压缩推进）



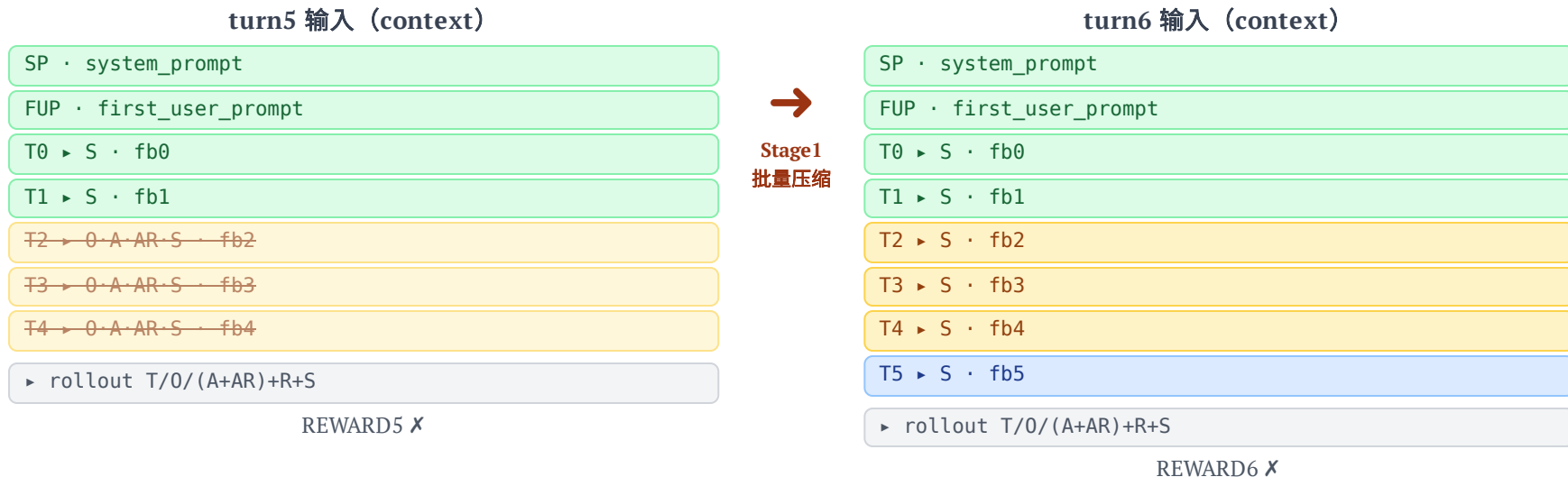
turn1 降级为 S；前缀延至 **T0:S** 后断裂，T2/T3 重算。recent-k=3 窗口（T2/T3/T4）保持全量。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算（删除线=被丢弃）

05 · SCRIBE 上下文管理机制 - turn5 → turn6（批量压缩）



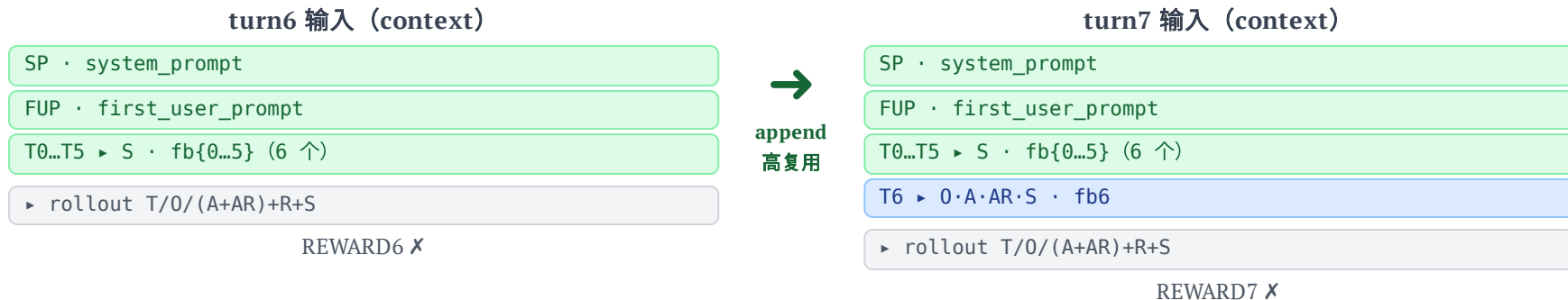
输入逼近上限 → **rollout** 前触发 Stage1 批量压缩：T2/T3/T4 的 O/A/AR 全折叠为 S；前缀在 T2 断裂，T3/T4 重算。原子 turn：压缩只发生在 turn 之间，不中断 turn 内 rollout。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算（删除线=被丢弃）

05 · SCRIBE 上下文管理机制 - turn6 → turn7（高复用）



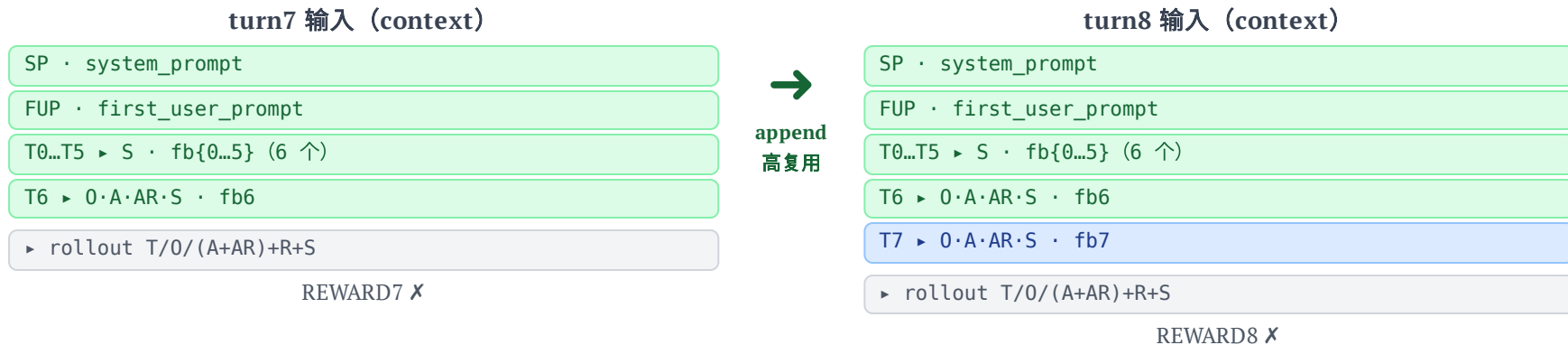
压缩后前缀短而稳定 → **整段 KV-cache 命中**，仅追加 turn6。SCRIBE 的 KV-cache 红利显现：每 turn 只为新增后缀付费。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn7 → turn8（高复用）



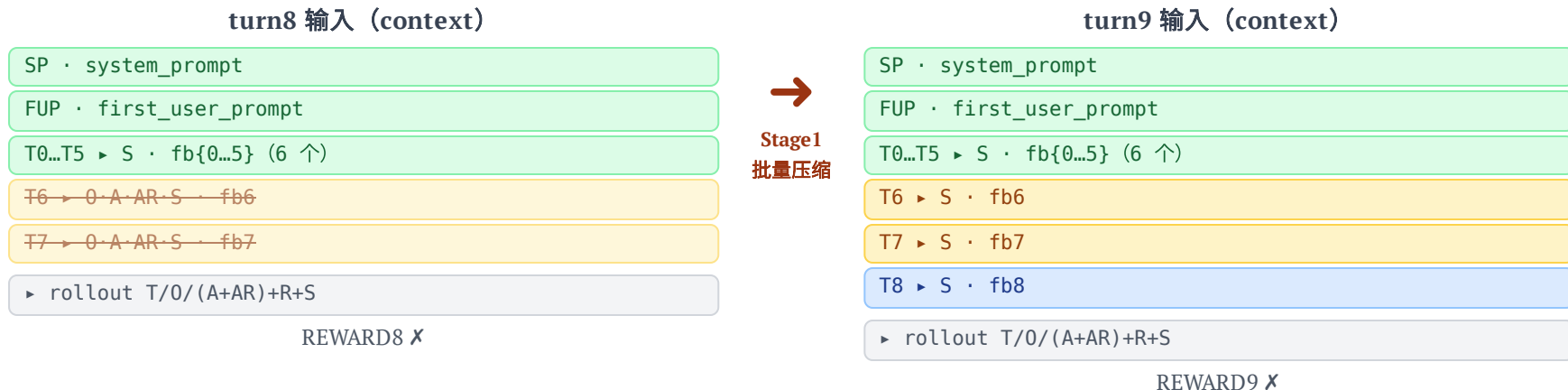
整段复用 + 追加 turn7；前缀越长，复用比例越高，单 turn 增量成本越小。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn8 → turn9（再次批量压缩）



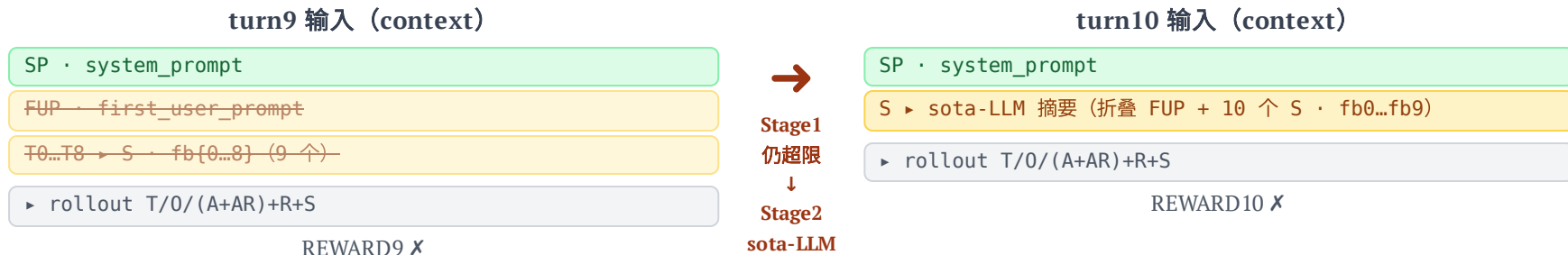
再次逼近上限 → Stage1 压缩 T6/T7 为 S；前缀在 T6 断裂。压缩是周期性的：每当 recent-k 窗口外累积过多 O/A/AR，就折叠一次。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn9 → turn10 (Stage1 → Stage2 双重压缩)



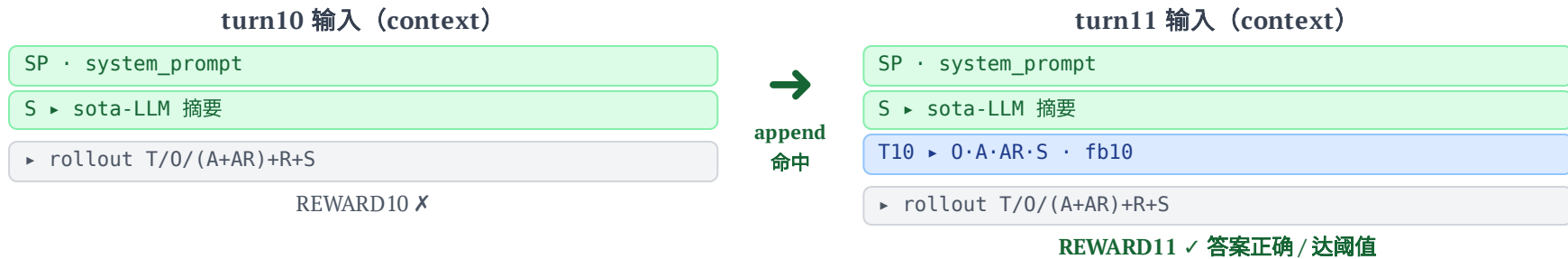
Stage1 收集 S 后仍超上限 → **Stage2** 调 sota-LLM 把 FUP + 所有 S + fb9 摘要成单个 <turn_summary>；前缀只剩 SP。仅此极端情形需一次 LLM 调用，绝大多数压缩是零成本的 Stage1。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn10 → turn11 (终止)



整段复用 + 追加 turn10 ; turn11 提交答案正确 (或 REWARD11 达阈值) → **loop 终止** ✓。整条轨迹: 12 个 turn, 仅 5 次压缩事件, 其余 6 次过渡为纯 append (KV-cache 命中)。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · 上下文机制定义（1/2） - Block 模型与压缩规则

Block 标签与顺序

T=think · A=tool_call · O=output · R=reflect · S=turn_summary ·

AR=tool_response

一个 turn 的 rollout = list[T/O/(list[A]+list[AR])] + R + S

step = 一次 LLM call；N 个 tool_call → N+1 步；最后非工具步产出 O+R+S

结束 turn

（注：rollout 实际输入始终排除 T&R——它们一次性；此处列出全部 block 是为训练时完整记录）

Block 层 vs 消息层

标签在 **block 层** 始终保留（供解析器切分 ScribeBlock）

消息层（模型下次实际读到的）：

- <think>/<reflect> 永不出现在未来输入（一次性）
- <turn_summary> 保留标签（recent 与压缩后 old 都留）
- <tool_call> 在无 S 的 old turn 降级为 O+A 时保留
- OUTPUT 裸文本；AR 由 role=tool 承载

压缩三规则

- ① **T&R 一次性**：生成后在下一步前剥离，永不进未来输入
- ② **recent-k 保留**：recent-k（默认 k=3）之前的 turn 只保留 S（标签完整）
- ③ **turn 原子性**：压缩只在 rollout 前触发（check-before-rollout），不在 turn 内中途触发——赌单 turn 不超长；首个大 turn 也难超当今 1M context

无 S 兜底

若某 turn 无 S（rollout 格式错）：

- 该 turn 老到只需保留 S 时 → 改保留 O + A
（O 作裸文本，A 带 <tool_call> 标签）
 - recent 但无 S 的 turn → 保留 O/A/AR
- 下文示例均假设每 turn 正确生成 S

05 · 上下文机制定义（2/2） - 压缩集合、两阶段与硬边界

两个集合（关键区分）

Token-count 集合（决定「是否压缩」）：整个输入 context = system_prompt + first_user_prompt + 所有 turn-feedback + 所有 input blocks，一起计数（**SP 也算**）

Compression 集合（决定「压缩什么」）：分阶段增长。**SP 永不压缩**（逐字保留——承载 task/protocol，须跨轨迹一致）

硬边界（Hard Boundary）

若**不可压缩前缀** = SP + FUP + 所有 feedbacks（所有 input block 已压成空）自身仍超上限 → 压缩无能为力
此时 **SCRIBE 抛错**：轨迹配置不可行（逐字保留部分单独溢出 context window）

两阶段压缩

Stage1（收集 S）：只动 input blocks，旧 turn 折叠为 S；FUP + feedbacks 逐字保留

Stage2（sota-LLM，当 S 序列仍接近上限时触发）：LLM 输入包含 FUP + feedbacks，一起折叠进单个 <turn_summary>；返回

`{"summary": "..."}` ，校验为恰好一个 turn_summary 块

仅 SP 逐字穿过两阶段；压缩直到不再接近上限才 rollout；单块仍超则重试 sota-LLM（其重试/退避与 SCRIBE 解耦）

check-before-rollout 纪律

压缩是 **turn 级**（非 turn 内级）：

- 整个 input blocks 的 token 接近上限 → rollout **前**自动触发压缩
 - 压缩到不再接近上限 → 才进入 rollout
 - 该 turn 最终存储：压缩后 input + 新生成/rollout blocks + reward
- 权衡：原子 turn 的 rollout 是对「该 turn 不至过长使 context 崩」的合理下注

06 · reward mechanism of a turn

16 个 metric (多维 reward 防 hacking) + 权重配比

06 · the reward mechanism of a turn - 16 metric 权重配比总结

M	metric	w	类别
M1	最终步奖励 answer+终止	0.52	核心
M2	工具使用质量	0.08	基础设施
M3	步数长度	0.02	效率
M4	格式正确性	0.10	守门员
M5	rollout 总 token	0.02	效率
M6	token 复用率	0.02	防抄袭
M7	忠实度 judge	0.02	summary 质量
M8	方向中立 judge	0.02	summary 质量
M9	turn 聚焦 judge	0.02	summary 质量
M10	流畅度 judge	0.02	summary 质量
M11	压缩比	0.01	防抄袭
M12	跨块 n-gram 重叠	0.03	防抄袭
M13	块内 n-gram 重叠	0.03	防抄袭
M14	工具参数合法	0.02	基础设施
M15	跨步重复工具	0.02	基础设施
M16	跨 turn 重复提交	0.05	防停滞
合计 Σw_i		1.00	clamp [0,1]

类别小计

- 核心 answer : M1 = **0.52**
- 守门员 format : M4 = **0.10**
- 基础设施 tool : M2+M14+M15 = **0.12**
- 效率 : M3+M5 = **0.04**
- summary 质量(judge) : M7-M10 = **0.08**
- summary 防抄袭(统计) : M6+M11+M12+M13 = **0.09**
- 防停滞 : M16 = **0.05**

设计哲学

- answer 主导但不过激 (0.52)
- 防抄袭指标为符号/反抄袭信号，非主驱动
- **多维 reward 防 hacking**
- M7-M10 合并一次 LLM 调用
- total = clamp($\Sigma w_i \cdot m_i$, 0, 1)

trajectory reward

```
trajectory_reward = mean(turn_reward) ×  
decay^len(turns)
```

用 mean 而非仅末 turn : 早 turn 坏行为直接受罚 ; 长度衰减奖励更少 turn 解决

07 · abstract of our contribution

可作为 A 类会议（如 ICLR）论文摘要

07 · abstract of our contribution

We present **SCRIBE**, a small yet foundational change to the ReAct loop for long-horizon **loop-engineering** tasks: at the end of every turn the agent explicitly writes a **Reflect (R)** and a **Turn-Summary (S)** of what it just did. This single change induces a new **context-management mechanism** — disposable T/R blocks, recent- k full retention, and automatic **Stage-1** (collect-S) / **Stage-2** (sota-LLM) compression — which in turn unlocks four benefits:

01

Painless context extension

Context grows across turns yet stays bounded, almost entirely by **zero-cost automatic compression** — only the rare Stage-2 needs one LLM call.

02

Token-level credit assignment

Each turn's summary is a first-class, rewarded block, so GRPO credit flows **trajectory → turn → step → token** — markedly more sample-efficient than broad-ReAct.

03

Higher accuracy & token efficiency

Both accuracy and token efficiency on loop-engineering tasks **exceed the pre-training model and the ReAct-trained model**.

04

High KV-cache reuse

Between consecutive turns only a small suffix is new while the **whole prefix is cache-reused**; compression is the only prefix-breaking event.

A small & beautiful change → a context-mechanism shift → many derived benefits.



谢谢聆听

Q & A

周浩洋 · JLU, SE · 2025.04.17 – Present